

Code Recommendations

1. Migrate Frontend to TypeScript

The frontend is currently written in plain JavaScript. For a codebase of this size, the lack of types makes refactoring risky and API contracts invisible — there is no way to know at write-time whether a component receives the right props or whether an API response has the expected shape.

- Migrate to TypeScript gradually — start with the API service layer and shared components, then cover pages over time.
- Define interfaces for all API request and response objects so the contract between frontend and backend is explicit and compiler-checked.
- Enable strict mode in tsconfig to catch the most common categories of bugs early.

2. Consolidate to a Single Frontend Technology

The project currently maintains two frontend technologies side by side: a legacy Blazor project and the current React application. This forces developers to context-switch between two completely different paradigms, and any shared logic ends up either duplicated or missing from one side entirely.

- Commit to React as the single frontend. No new features should be built in Blazor.
- Plan the retirement of the Blazor project so the team can stop maintaining two frontends in parallel.

3. Introduce a Service Layer in the Backend

The backend currently has no service layer — all business logic lives directly inside controllers. This means a single controller action is simultaneously responsible for receiving HTTP requests, enforcing business rules, writing to the database, sending emails, handling files, and generating audit logs. Controllers become very large and any change risks unexpected side effects.

- Introduce a service layer between controllers and the data layer. Controllers should only handle HTTP concerns: parse the request, call a service, return the response.

- Business logic (status transitions, number generation, email dispatch, audit logging) belongs in services, not controllers.
- Services should be registered via dependency injection, making them straightforward to swap, mock, and test in isolation.
- Logic that currently exists as copy-pasted blocks across multiple methods should live in a single dedicated service with one reusable method.

4. Fix Naming Conventions

There are several naming inconsistencies across the codebase — typos baked into property names, inconsistent casing, and vague method names. Because these names flow through models, database columns, DTOs, and API responses, they create confusion at every layer.

- All C# properties should follow PascalCase consistently — no camelCase or all-lowercase property names on entities or DTOs.
- Typos in existing property names should be corrected with a coordinated rename across models, DTOs, migrations, and any frontend references that depend on those field names.
- Controller action methods should follow PascalCase and be written as proper compound words (e.g. AddWorkOrder, not Addworkorder).
- Avoid naming methods or endpoints with numeric suffixes like GetList2. If two versions exist, one should be removed, or both should have names that describe what actually differs between them.

5. Eliminate Code Duplication

Blocks of logic that are copy-pasted in multiple places are a maintenance liability. Every time a bug is fixed or a requirement changes, the same change needs to be applied to every copy — and it is easy to miss one.

- Any logic that appears more than once should be extracted into a shared method or service.
- If two endpoints do nearly the same thing with minor differences, consider whether one can be removed, or whether both can delegate to a single shared method with parameters.
- The same principle applies on the frontend — repeated data-fetching patterns, repeated form logic, and repeated UI structures should be abstracted into shared hooks or components.

6. Centralize Frontend API Communication

When HTTP requests are made in different ways across different parts of the frontend, concerns like authentication headers, error handling, and base URL configuration need to be maintained in many places. Any change requires touching many files.

- All API communication should go through a single centralized service. Authentication headers, error handling, and base URL configuration are defined once and applied everywhere.
- Individual pages and components should call this service, not make HTTP requests directly.

7. Fix the N+1 Query Problem

An N+1 problem occurs when a list endpoint executes one query to fetch N records and then fires an additional query for each record to fetch related data. Loading a list of 20 items hits the database 21 times instead of once. This is a silent issue — it works fine with small data volumes but degrades noticeably as data grows.

- Related data needed for list responses should always be loaded as part of the main query — via a JOIN or a single subquery — rather than inside a loop or projection.
- Review all list endpoints for this pattern and correct them before data volume makes the degradation visible in production.

8. Keep Filtering and Sorting in the Database

Loading entire database tables into memory and then filtering the results in application code does not scale. The memory and time cost grows with the size of the table regardless of how small the final result set is.

- All filtering, sorting, and pagination should be expressed as database-level operations. SQL should do the work, and only the matching rows should be returned to the application.
- As a rule: filtering and sorting must always come before materializing the query into a list — never after.
- Return only the fields the caller actually needs. Endpoints used for dropdowns or summaries should not return full entity objects with all their properties and relations loaded.

9. Manage Secrets Properly

Credentials such as database passwords and third-party API keys should never be committed to source control. Once a secret is in git history it is effectively compromised — it can be found in clones, forks, and history even after deletion from the latest version.

- All secrets must be stored in environment variables or a dedicated secrets manager.
- Configuration files committed to the repository should contain only non-sensitive defaults and placeholders — never real credentials.
- Any secrets that have already been committed should be considered compromised and rotated immediately.

10. Add Automated Tests

There are currently no automated tests in the project. Every change relies entirely on manual verification, and regressions are only discovered in production. This risk compounds as the codebase grows.

- Start with unit tests for the service layer once it is introduced — services contain the business logic and have no HTTP dependencies, making them the natural and highest-value place to begin testing.
- Add component tests for shared frontend components.
- Even a small suite of integration tests covering the most critical user flows would significantly reduce the risk of deploying broken changes.